

TorchFDTD: GPU-accelerated finite-difference time-domain simulation with discrete adjoints and host-streamed execution for photonic inverse design

Hyoseok Park

Department of Physics, Chungnam National University, Daejeon, Republic of Korea

23 September 2026

Abstract

Gradient-based photonic design needs full-wave derivatives with respect to millions of parameters, but on a single GPU workstation the time-domain adjoint is limited by device memory and by the incompatibility of fused update kernels with automatic differentiation. Here, we present TorchFDTD, an open-source finite-difference time-domain (FDTD) package that addresses both limits. The Yee update, the convolutional perfectly matched layer and the auxiliary differential equations of dispersive media execute as fused CUDA kernels captured in a CUDA graph, and every kernel is paired with a hand-written transpose, so material and geometry derivatives are obtained by a discrete adjoint that PyTorch chains with differentiable objectives built from the supported observations. The forward states required by the adjoint are supplied by checkpoint replay or, for lossless periodic problems, by time reversal. A streamed mode advances causal slabs of the domain one block at a time and keeps the global state and the checkpoints in host memory, which lowers the device allocation while preserving the resident discretization. Guided-mode ports, near-to-far projection, GDSII import and a browser workbench complete the design workflow. We validate the package against analytic solutions, against Meep, FDTDx and a rigorous coupled-wave solver, and against automatic differentiation and finite differences. On an RTX 5880 the fused path completes full forward solves of eight test scenes 9.6 to 17.1 times faster than the PyTorch FDTD package it extends, and host streaming lowers the peak device allocation of a 256^3 adjoint by 56% at a 4.1 to 4.8 fold cost in time. A 54-million-cell pillar-array lens coupled to an angular-spectrum objective yields an adjoint derivative within 0.78% of a central difference on an RTX 3060. The time-domain adjoint of a device with tens of millions of cells thus becomes available on a single workstation GPU.

Keywords: finite-difference time-domain, adjoint method, inverse design, GPU computing, PyTorch, metasurfaces, nanophotonics

Program summary

Program title: TorchFDTD

Developer’s repository link: <https://github.com/hyoseokp/TorchFDTD>

Licensing provisions: MIT

Programming language: Python 3.10 or later, with CUDA C++ kernels compiled at run time through CuPy and a JavaScript browser workbench

Supplementary material: the validation records behind every figure and generated table, the two scripts that typeset them from those records, and SHA-256 provenance manifests

Nature of problem: Gradient-based design of nanophotonic devices requires a full-wave Maxwell solver that returns broadband observables and their derivatives with respect to millions of material or geometry parameters. On a single workstation the time-domain adjoint is limited

by GPU memory, because the backward sweep needs the forward field history and the working field state must fit the device. A second limit is the incompatibility between fused CUDA update kernels and framework automatic differentiation.

Solution method: The Yee scheme with convolutional perfectly matched layers, auxiliary-differential-equation dispersion and anisotropic tensors runs as two fused CUDA kernels per step inside a CUDA graph. Each kernel has a hand-written transpose, and the discrete adjoint is exposed to PyTorch as a custom autograd function so that geometry parameterizations, objectives built from the supported observations and optimizers are ordinary Torch code. Forward states for the transposed sweep come from binomial checkpoint replay or time reversal. A host-streamed mode partitions the domain into causal slabs with halos, advances one slab for K steps on the device and keeps the global state and checkpoints in host memory. Guided-mode sources and monitors, total-field/scattered-field boxes, near-to-far projection, angular-spectrum propagation, GDSII import and process-level or tensor batches are provided.

Additional comments including restrictions and unusual features: Fused kernels require an NVIDIA GPU with CuPy, and a CPU path executes the same discrete operators for reference. Host streaming keeps full-domain state banks in host memory, so host capacity and device–host bandwidth bound the usable problem size and speed. Independent lateral tiling is an approximation that changes the boundary-value problem and is reported with its error against a full-domain solution. Multi-GPU execution of one domain is not part of this release.

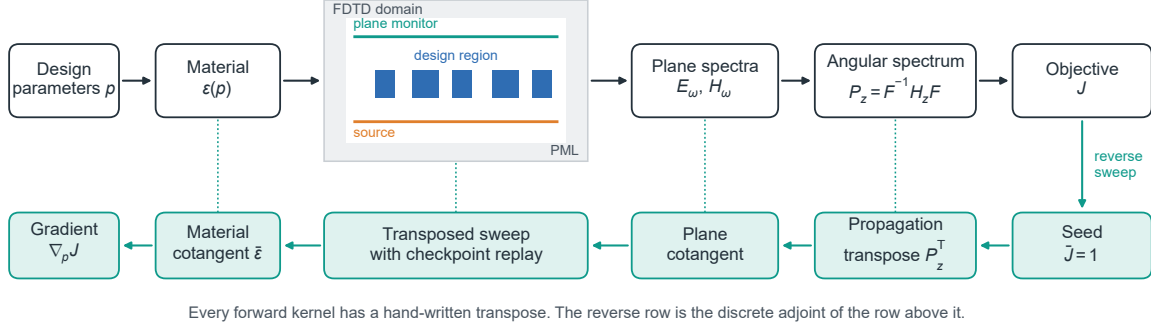
1 Introduction

The finite-difference time-domain (FDTD) method advances Maxwell’s equations on a staggered grid and returns broadband observables from a single run [1, 2]. Absorbing layers [3, 4], auxiliary differential equations for dispersive media [5] and subpixel material averaging [6] made it the standard full-wave solver of nanophotonics, and packages such as Meep [7] and commercial tools [8, 9] expose it through scripting interfaces. Inverse design changed what is asked of the solver. Adjoint sensitivities turn one additional simulation into the gradient of an objective with respect to millions of design variables [10–16], and design frameworks built on frequency-domain solvers [17, 18] or on hybrid time and frequency-domain evaluation [19] made that gradient central to nanophotonic design. The practical question became how to obtain it quickly on the hardware that a design group owns. Automatic differentiation of a Maxwell solver written in a machine-learning framework answers part of that question [20]. The Fourier modal solver TORCWA showed that a PyTorch implementation makes rigorous coupled-wave gradients available to ordinary optimizers on a graphics processor [21]. GPU time-domain solvers such as fdtd-z moved the Yee update itself onto the device [22], and FDTDx brought reverse-mode differentiation and multi-GPU sharding to time-domain simulation in JAX [23].

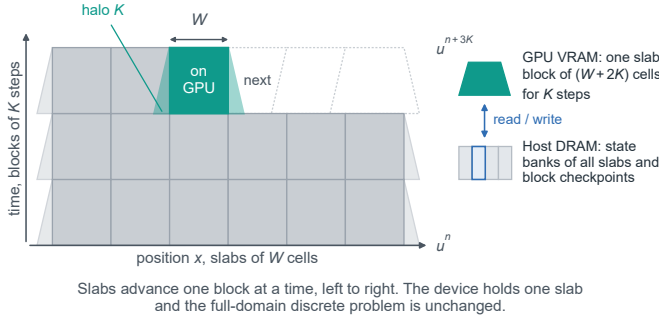
Two costs remain for the time-domain adjoint on a single workstation. The first is memory. The backward sweep needs the forward field history, and time-reversible reconstruction or checkpoint replay trades retained history against recomputation [23–25]. However, even with that history under control, a resident implementation must hold the complete working state on the device. For a 10^9 -cell grid in single precision the six field components alone occupy 24 GB, and the adjoint sweep adds a cotangent state of equal size and at least one checkpoint, so the working state exceeds 70 GB before the material arrays and absorber memories are counted. That is more than the 12 and 48 GB of the two GPUs used in this work. The second cost is the interaction between differentiation and kernel fusion. Framework automatic differentiation records every array operation, so the fused update kernels that give an FDTD code its speed cannot participate in the graph unless they are supplied with their own derivative rules.

Here, we address both costs in TorchFDTD. Its Yee, absorber and dispersion updates run as fused CUDA kernels captured in a CUDA graph. Instead of letting the framework record these

a Optical model and its discrete transpose



b Host-streamed sweep over causal slabs



c Independent lateral tiles

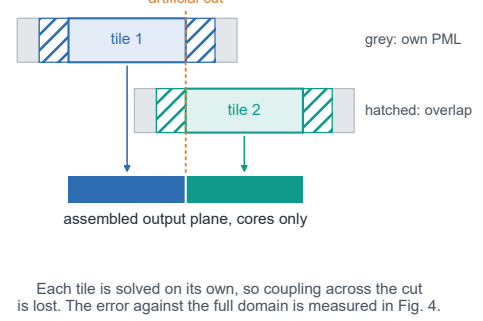


Figure 1: Execution and differentiation in TorchFDTD. (a) The forward map from design parameters to an objective evaluated after angular-spectrum propagation, and its transposed chain. Every forward kernel has a hand-written transpose, and the transposed sweep obtains forward states from checkpoint replay. (b) Host-streamed execution sweeps causal slabs of W cells with halos of K cells over blocks of K steps. One slab block resides on the GPU while the state banks of all slabs and the block checkpoints stay in host DRAM. (c) Independent lateral tiles solve overlapping subdomains with their own absorbers and assemble the core fields on the output plane. Coupling across the artificial cut is lost, and the resulting error is measured against a full-domain solution.

updates, we give each kernel its own derivative rule, a hand-written transpose. The discrete adjoint is therefore exposed to PyTorch as a custom autograd function and chains with Torch geometry parameterizations, optimizers and differentiable objectives built from the supported observations. We partition the field state into causal slabs, strips of the domain padded by halos as wide as the number of steps advanced per block. A streamed mode advances one slab at a time on the device while the global state and the checkpoints reside in host memory (Fig. 1). The device allocation then follows the width of one slab rather than the length of the domain along the partitioned axis, although it still grows with the transverse cross-section. The discrete problem is unchanged. Guided-mode ports, total-field/scattered-field boxes, near-to-far projection, angular-spectrum propagation to remote observation planes, GDSII import and a browser workbench connect the solver to a complete design workflow. The grid and backend foundation come from the MIT-licensed flaport/fdtd package [26], and PyTorch supplies device arrays, graph capture and the optimizer interface [27].

On the tested scenes the fused path completes full forward solves 9.6 to 17.1 times faster than the package it extends, and host streaming lowers the peak device allocation of a 256^3 adjoint by 56% at a 4.1 to 4.8 fold cost in time.

This paper describes the numerical method (Section 2), the discrete adjoint (Section 3), the execution modes (Section 4) and the software structure with usage examples (Section 5). Section 6 validates the package against analytic solutions, independent solvers and finite differences, and Section 7 reports throughput, the memory–time trade-off of host streaming and a propagated optical objective on a workstation.

2 Numerical method

2.1 Yee update and time step

The six field components occupy the standard staggered positions [1]. In a nonmagnetic medium the reduced update reads

$$E^{n+1} = E^n + S \epsilon^{-1} \mathcal{C}^-(H^{n+1/2}) + s^{n+1}, \quad H^{n+3/2} = H^{n+1/2} - S \mathcal{C}^+(E^{n+1}) + m^{n+3/2}, \quad (1)$$

where $S = c\Delta t/h$ with h the smallest mesh step, $\mathcal{C}^\pm$ are the forward and backward difference curls scaled by the local metric, and s and m are soft electric and magnetic source increments added after the corresponding update. Fields start from zero, the first recorded electric sample lies at Δt and the first magnetic sample at $3\Delta t/2$, and every Fourier transform in the package keeps that half-step offset. For d active dimensions with minimum primal widths h_a the time step is bounded by

$$\Delta t \leq \frac{\eta}{c \sqrt{\sum_{a=1}^d h_a^{-2}}}, \quad 0 < \eta \leq 0.99, \quad (2)$$

which reduces to $\eta h/(c\sqrt{d})$ on a uniform mesh. A graded rectilinear mesh keeps this fine-grid step, so coarsening the background reduces the cell count at a fixed physical duration without local time stepping. The two-dimensional solver suppresses the z derivatives while keeping all six components, and Bloch problems carry complex fields.

2.2 Boundaries

Convolutional perfectly matched layers [4] store one memory variable per transverse derivative inside each absorbing face. The stretching coefficients κ , σ and α follow polynomial profiles configured per face, so opposite faces and neighboring faces of different type may carry different depths and gradings. Periodic and Bloch pairs wrap the difference operators without a duplicated end plane, and the Bloch phase enters through the wrapped stencil. Perfect electric conductor faces and electric antisymmetry planes hold the tangential electric field at zero on the boundary node. Perfect magnetic conductor and magnetic symmetry faces need the dual arrangement, in which the upper boundary node of each tangential electric component becomes a stored degree of freedom. The package carries those face arrays through the resident, streamed and batched paths, through their checkpoints, and through the absorber memories of neighboring faces. Any face type may adjoin any other, so Section 6 measures the reflection of an absorbing face next to a magnetic wall.

2.3 Materials and geometry

A nondispersive cell carries a real permittivity at each Yee electric position. Dispersive materials add up to sixteen Drude or Lorentz poles integrated with the trapezoidal auxiliary differential equation of Okoniewski et al. [5], and measured refractive-index tables are fitted to a passive multipole model whose error and band are reported before the fit is accepted. Anisotropic media [28] use a symmetric positive-definite permittivity tensor sampled at grid nodes and applied to the curl through the node assembly $S = D^{-1/2}(\frac{1}{8} \sum R^\dagger \epsilon^{-1} R) D^{-1/2}$, with the tensor restricted to eigenvalues at or above one so that the vacuum time step of Eq. (2) remains admissible. Inside an absorbing layer the stretched-coordinate formulation is unstable for tensors whose principal axes are rotated relative to the face normal or whose normal eigenvalue is the intermediate one, a consequence of the backward-wave criterion of Bécache et al. [29] that also constrains the corrected unsplit layers of Oskooi and Johnson [30]. TorchFDTD therefore admits a tensor inside a face only when the face normal is a principal axis with a non-intermediate eigenvalue, a

rule checked node by node. Dense one-step spectral radii on small Bloch grids equal one to nine decimal places for admitted tensors and lie between 1.0049 and 1.0089 for rejected ones, and a 4000-step sweep over fifteen admitted cases with contrasts up to sixteen shows no growth. The rejected cases grow by factors of 7.9×10^3 and 7.6×10^{12} over that sweep.

Geometry enters through analytic solids (boxes, cylinders, spheres, ellipsoids, rings and extruded polygons with holes) placed by an ordered rotation and a pivot, with a lower mesh order winning overlaps. Membership is evaluated only inside a conservative axis-aligned support, and an optional dielectric subpixel operator averages the permittivity across interfaces [6]. GDSII layouts map layer and datatype pairs to extrusion intervals and materials, apply Boolean etch layers, staircase a sidewall angle on the native z nodes and keep polygon holes with even-odd filling [31].

2.4 Sources, monitors and projections

Point and sheet sources inject Gaussian, ramped continuous or sampled waveforms with Cartesian or angular polarization. A one-way plane covering a periodic transverse cell injects a normally incident wave without a backward component, and a closed total-field/scattered-field box [32] surrounds an isolated scatterer with an incident field that is corrected on its faces. Guided-mode sources solve the full-vector finite-difference eigenproblem of the port cross-section [33] at the temporal wavenumber $\tilde{k}_0 = 2 \sin(\omega \Delta t / 2) / (c \Delta t)$, convert the eigenvalue to the longitudinal Yee propagation constant $\beta = 2 \arcsin(\tilde{\beta} \Delta w / 2) / \Delta w$, and inject electric and magnetic Huygens sheets at their staggered positions and times. The eigensolver returns a canonical basis for degenerate polarization pairs by diagonalizing the overlap of their transverse components, so a port's mode index is stable across machines.

Point monitors record one component per step. Plane monitors accumulate six-component complex spectra with the Yee interpolation and the magnetic half-step phase applied before the signed Poynting flux is formed, and flux ratios are taken against a matching reference run. A closed six-face box of stored spectra feeds a near-to-far transform of the equivalent surface currents [34] in the asymptotic form and in a finite-distance form built on the free-space dyadic Green function. The transform accepts angular, spherical, Cartesian and direction-cosine observation sets and a complex passive exterior index that may vary per frequency, and an open-surface variant projects from one to five faces with an edge window. Periodic cells return Bloch diffraction orders separated into forward and backward propagating branches.

3 Discrete adjoint

3.1 Transposed updates

Let u^n collect the field, absorber and polarization state after step n , and let the forward step be $u^{n+1} = F_n(u^n, p)$ with parameters p . For an objective $J(u^N, \dots)$ the reverse recursion

$$\bar{u}^n = \left(\frac{\partial F_n}{\partial u} \right)^\top \bar{u}^{n+1} + \frac{\partial J}{\partial u^n}, \quad \bar{p} = \sum_n \left(\frac{\partial F_n}{\partial p} \right)^\top \bar{u}^{n+1}, \quad (3)$$

returns the gradient of J with respect to every parameter after one backward sweep. Each transpose is a hand-written kernel that mirrors its forward kernel. The Yee curls transpose to the opposite-direction differences with the same metric. The absorber recursion transposes to $\bar{\psi}_{\text{mem}} = \bar{\psi} + \bar{y}$, $\bar{D} = \kappa^{-1} \bar{y} + c \bar{\psi}_{\text{mem}}$ and $\bar{\psi}_{\text{old}} = b \bar{\psi}_{\text{mem}}$ for the memory variable ψ with coefficients b and c , and the trapezoidal dispersion update transposes to the same two-by-two system applied to the cotangents. The material cotangent $\bar{\epsilon}^{-1}$ accumulates the product of the electric cotangent with the curl at every step.

A chain of differentiable material producers maps that cotangent onto design variables. The variables may be a scalar or tensor permittivity per cell, a density field passed through a physical-radius filter, symmetry averaging and a smooth projection [35, 36], an analytic solid whose occupancy is a smooth signed-distance step of fixed physical width, a polygon or spline outline whose vertices or control points move that step, or a source waveform. Objectives may combine point signals, plane spectra, the amplitudes of guided modes at any number of ports and the far or near-zone fields of a projected box, since each observation is linear in the stored fields and carries its own transpose. Because the transposed sweep is wrapped as a Torch autograd function, everything upstream of the permittivity and downstream of the observations remains ordinary differentiable Torch code.

3.2 Checkpoint replay and reversibility

Retaining u^n for every time step scales with both volume and duration, so the backward sweep instead replays the forward pass between checkpoints. With C checkpoints and N steps the binomial schedule of Griewank and Walther [24] minimizes the number of recomputed steps, and the checkpoints are placed in GPU or host memory according to the execution policy. A retained backward pass keeps the adjoint state between transposes, so several objectives can be differentiated from one forward run. For lossless nondispersive periodic problems an opt-in reversible mode reconstructs the forward fields by stepping Eq. (1) backward, in the spirit of Mahlau et al. [23], and the absorbing-face variant records the fields at the interface between interior and absorber so that the interior reconstruction stays lossless while the absorber is replayed. Which mode is admissible follows from the material and boundary content of the project rather than from a user setting.

4 Execution on a workstation

4.1 Fused kernels and batches

One simulation on the device runs as a sequence of fused kernels compiled through CuPy [37]: one kernel updates all electric components together with the absorber memories of every face, and one kernel updates the magnetic components. The kernels are captured once into a CUDA graph together with source injection and monitor accumulation, so a run of N steps replays N graph launches without Python in the loop [27]. The CPU path executes the same discrete operators with NumPy and Torch tensors and serves as the reference for every device test.

A design loop evaluates many structures, so the package runs them either as separate processes or as one tensor batch. A process-level batch runner executes independent projects in separate processes on one or more devices with checksum-validated resumption, per-case error capture and a memory estimate that limits the number of resident cases. A tensor batch instead places B compatible structures along a leading batch axis of the field arrays and updates all of them in one launch. For small grids, this single launch amortizes the kernel-launch and graph overhead. Because structures with different meshes or durations cannot share an axis, they are grouped into cohorts of identical shape and executed cohort by cohort. The results return in the input order. A seeded differential-evolution optimizer [38] drives either level through a user-defined objective for gradient-free design.

4.2 Host-streamed slabs

The streamed mode partitions the domain along x into slabs of W cells and advances each slab for K steps before moving to the next, so a block of K steps needs a halo of K cells on either

side of the slab (Fig. 1b). Only the slab under update, its halos and its checkpoint reside on the device. The remaining field state lives in banks, contiguous buffers in host DRAM that are read into and written from the device slab by slab. The transposed sweep runs this tiling in reverse, and both sweeps see the same number of field updates plus the halo overhead $2K/W$ per block. Absorber memories, dispersion states and the stored magnetic-wall faces travel with their slabs, and the slab whose core ends at the upper boundary owns the face arrays. Because the halos carry the complete state that the Yee stencil needs at every block, the streamed sweep reproduces the resident discrete problem up to floating-point ordering.

A policy is admitted only after accounting for the active device workspace, the global state banks, the checkpoints, the material tensors and the observation buffers. Slab width W and temporal depth K control the active field extent $W + 2K$, while the checkpoint count controls retained history and replay work. For a domain of $N_x \times N_y \times N_z$ cells partitioned along x , the device workspace therefore scales with $(W + 2K)N_yN_z$ rather than with $N_xN_yN_z$, so it still grows with the transverse cross-section. The host allocation scales with the full volume plus the retained checkpoints, and host capacity and device–host bandwidth bound the usable policies.

4.3 Lateral tiling and exterior propagation

A separate interface partitions a planar device into independently solved, overlapping lateral tiles, each with its own absorber and output plane, and assembles their core fields on the global plane (Fig. 1c). In contrast to the causal slabs above, this is an overlapping-domain approximation of the kind used for large-area metasurfaces [39, 40], and it changes the boundary-value problem. A neighboring-tile mismatch is reported as a diagnostic, and Section 6.4 measures the error against a full-domain solution.

In a homogeneous, lossless and source-free exterior containing outgoing waves, a recorded plane field is propagated with the angular spectrum [41],

$$u(x, y, z_0 + d) = \mathcal{F}_{xy}^{-1} \left[\mathcal{F}_{xy} \{u(x, y, z_0)\} \exp \left(i d \sqrt{k^2 - k_x^2 - k_y^2} \right) \right], \quad d \geq 0, \quad (4)$$

where the square-root branch makes evanescent components decay. The implementation propagates the sampled electric and magnetic components rather than a scalar transmission coefficient, and zero padding limits the periodic wraparound of the discrete transform [42]. The interface returns selected planes, axial sections or points without storing an exterior volume. For a tiled plane $u = \sum_j R_j u_j(p)$ with fixed assembly maps R_j , the reverse operation applies $R_j^\top$ to the global plane cotangent before the local FDTD adjoint, and the propagation of Eq. (4) has a differentiable transpose, so an exterior intensity objective differentiates to the material parameters.

5 Software structure and usage

TorchFDTD is a Python package of about 36 000 lines with a serializable scene model, a solver core, an adjoint layer and a browser workbench. A Project holds the region, materials, structures, sources, monitors and execution policy, and it round-trips through JSON so that the workbench, the command line and scripts operate on one description. The solver core contains the fused CUDA kernels and their transposes, the streamed executor, the batch runners and the CPU reference path. The adjoint layer wraps the transposed sweep as Torch autograd functions and provides the material producers, mode-port and projection objectives, memory estimation and checkpoint tiers. The workbench is a FastAPI service with a JavaScript client for scene editing, execution, field inspection and GDSII import. Optional extras add CuPy for the fused kernels, gdstk for layouts and h5py for HDF5 output, and the repository carries 1551 tests that exercise the CPU path everywhere and the CUDA path where a device is present.

```

from torchfdtd import (Project, Region, Structure, Source, Monitor,
                        Simulation)

project = Project(
    name="My waveguide",
    region=Region(dimension="3d", size=(8, 6, 2), mesh=0.05, steps=1000,
                  backend="cuda", cuda_kernel="fused"),
    structures=[Structure(name="core", size=(8, 0.65, 0.4))],
    sources=[Source(center=(-2.5, 0, 0), wavelength=1.55)],
    monitors=[Monitor(name="output", center=(2, 0, 0))],
)
project.save("project.json")          # opens in the browser workbench
result = Simulation(project).run()
result.save("results/run.npz")

```

Listing 1: A forward simulation with the fused CUDA path.

```

import torch
from torchfdtd import (Project, Region, Source, Monitor, AdjointOptions,
                        DifferentiableSimulation, smooth_sphere_epsilon)

project = Project(
    region=Region(size=(1.6, 1.5, 1.4), mesh=0.1, pml_cells=3, steps=40,
                  precision="float64"),
    sources=[Source(center=(-0.2, 0, 0), pulse="continuous",
                    wavelength=1.1)],
    monitors=[Monitor(center=(0.1, 0, 0))],
)
radius = torch.nn.Parameter(
    torch.tensor(0.25, device="cuda", dtype=torch.float64))
model = DifferentiableSimulation(
    project, AdjointOptions(checkpoints=4, storage="host",
                            host_budget_bytes=128 * 1024**2))
optimizer = torch.optim.Adam([radius], lr=0.003)
optimizer.zero_grad()
epsilon = smooth_sphere_epsilon(project.region, radius, inside=3.0, width=0.09)
result = model(epsilon)
loss = result.signals[:, 0].square().mean()
loss.backward()
optimizer.step()

```

Listing 2: A differentiable simulation with checkpoint replay.

Listing 1 runs a three-dimensional waveguide with a fused CUDA solve and saves the result. Lengths are in micrometers.

Listing 2 differentiates a point-signal objective with respect to the radius of a smooth sphere. The permittivity is built by a differentiable producer from a Torch parameter, the differentiable simulation replays four host checkpoints during the transposed sweep, and a standard optimizer consumes the gradient. Replacing `AdjointOptions` by a streamed policy with a slab width and temporal depth switches this model to host-streamed execution.

Each measurement in this paper is a JSON record under `docs/validation` with the hardware, execution settings and source hashes of its run. The figures and tables are typeset from these records by scripts that run no solver, and a provenance manifest lists the hash of every record they read.

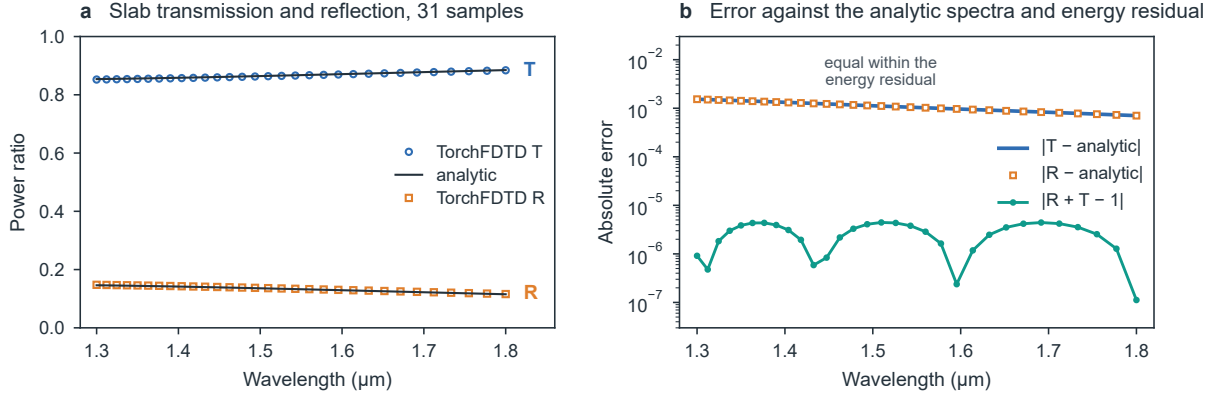


Figure 2: Plane-monitor spectra of the lossless slab. (a) Transmission and reflection of the 31 recorded samples, with Eq. (5) as the solid line. (b) Absolute errors of both spectra and the energy residual $|R + T - 1|$ on a logarithmic axis.

6 Validation

6.1 Analytic references

A lossless slab of index 1.5 and thickness $0.2 \mu\text{m}$ in air, excited by a broadband sheet in a periodic two-dimensional cell with a $0.025 \mu\text{m}$ mesh and 16 absorbing cells at each end, is compared with the normal-incidence transmission

$$T(\lambda) = \left[1 + \left(\frac{n^2 - 1}{2n} \right)^2 \sin^2 \left(\frac{2\pi n d}{\lambda} \right) \right]^{-1}, \quad R = 1 - T, \quad (5)$$

at 31 wavelengths between 1.3 and $1.8 \mu\text{m}$. The plane-monitor spectra reach maximum absolute errors of 0.00153444 in transmission and 0.00153535 in reflection, and the residual $R + T - 1$ stays below 4.43991×10^{-6} (Fig. 2). The transmission and reflection errors are equal and opposite and three orders of magnitude above the energy residual, so the remaining error lies in the discrete response of the slab rather than in the flux monitors.

A closed total-field/scattered-field box around a dielectric sphere yields the scattering cross-section over nine wavelengths, which is compared with the Mie series [43]. The error is set by the staircased sphere surface rather than by the box or the duration, and it is not monotonic in the mesh. It falls from 8.9% at a $0.1 \mu\text{m}$ mesh to 0.31% at $0.05 \mu\text{m}$, rises to 1.05% at $0.025 \mu\text{m}$ and falls again to 0.55% at $0.02 \mu\text{m}$, while doubling the physical duration leaves the $0.025 \mu\text{m}$ value unchanged (Table 1, Fig. 3). Cross-sections of this sphere are therefore accurate to about one percent from a $0.05 \mu\text{m}$ mesh onward, where the staircase rather than the resolution limits the error.

Radiation projections are checked against the closed-form fields of an electric dipole. With 14, 28 and 56 samples per face axis the finite-distance projection reaches relative errors of 4.6×10^{-3} , 1.1×10^{-3} and 2.8×10^{-4} at five points between 0.55 and $1.35 \mu\text{m}$ outside the box, a second-order sequence. The asymptotic and finite-distance models agree to 1.9×10^{-4} at a radius of $2000 \mu\text{m}$. A lossy exterior with index $1.3 + 0.05i$ reproduces the analytic near and far fields with the same convergence. For an open surface the single-face projection of a Gaussian-apodized directional emitter converges to the closed-box result as the field on the omitted faces decays, from 9.6×10^{-2} to 3.8×10^{-3} as the beam waist grows from 0.4 to 0.8 wavelengths, whereas an isotropic dipole stays at 0.79 under refinement. The closed-box projection is therefore accurate from just outside the box to the far field, while an open surface suits directional emitters whose field on the omitted faces is small.

Table 1: Closed-box TFSF sphere scattering on RTX 5880. The maximum relative cross-section error is taken over nine wavelengths. Physical domain and PML thickness stay fixed. The final row doubles the physical duration. The error sequence is not monotonic.

h (μm)	Grid	Time (fs)	Max error (%)	Empty box (μm^2)
0.1	32^3	120	8.8963	6.68×10^{-15}
0.05	64^3	120	0.3086	1.11×10^{-14}
0.025	128^3	120	1.0474	2.19×10^{-15}
0.02	160^3	120	0.5514	1.37×10^{-14}
0.025	128^3	240	1.0474	2.32×10^{-15}

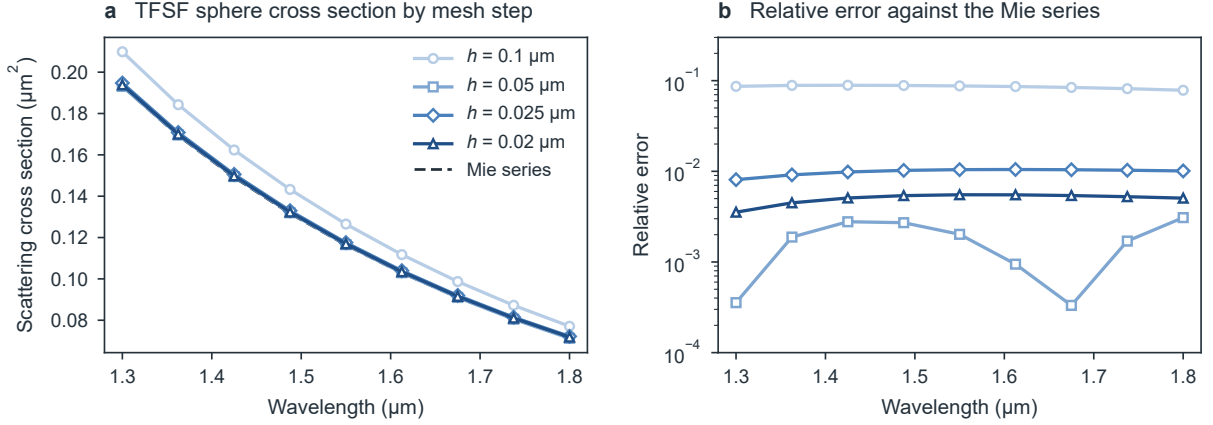


Figure 3: Scattering cross-section of the dielectric sphere from the closed total-field/scattered-field box at four mesh steps. (a) Recorded cross-sections with the Mie series as reference. (b) Relative error against the Mie series at the nine recorded wavelengths.

Guided-mode ports are validated on a straight guide, an interface between guides of index 1.0 and 1.2, a symmetric Y branch and a four-port crossing, and the complex scattering parameters of the interface are compared with an independent discrete interface solution. A narrow-aperture port on a wide domain matches the full-cell network to 1.6×10^{-5} in the scattering parameters. The Y branch obeys reciprocity to 2.0×10^{-3} with a 1.2×10^{-4} arm asymmetry, and the crossing obeys reciprocity to 2.4×10^{-4} with a 1.2×10^{-7} gap between its four-fold rotated columns. The port model is therefore reciprocal and symmetric to 2×10^{-3} or better, and the power missing from the guided channels of the Y branch is radiated by the staircased junction, a property of the discretized geometry.

The anisotropic path is checked against a birefringent slab of principal indices 1.5 and 2.0 in a co-aligned exterior of indices 1.2 and 1.4 that fills both absorbing faces along the propagation axis. The two polarization channels reproduce the analytic transmission to 1.37% and 0.33%, and the derivative of the transmission with respect to the slab index agrees with the analytic derivative to 6.9%. A magnetic wall next to an absorbing face reflects 1.7×10^{-4} of the incident power with the default profile and 3.5×10^{-4} with $\kappa = 3$, $\alpha = 0.05$ and a quadratic grading. On this mesh anisotropic transmission is thus accurate to about one percent and its index derivative to about seven percent, and a magnetic symmetry plane can adjoin an absorber at a reflection cost below 4×10^{-4} of the incident power.

Table 2: Recorded gradient agreement of the discrete adjoint. Relative errors compare the transposed sweep with full Torch automatic differentiation unless a finite-difference (FD) reference is named. FP64 on CPU unless stated.

Path	Reference	Relative error
Polygon solid, checkpointed adjoint	autograd, CUDA FP32	1.2×10^{-7}
Magnetic-wall faces with CPML	autograd	2×10^{-16}
Dispersive faces, four pole parameters	autograd	2.7×10^{-16}
Polygon vertices, voxel VJP	central FD, CPU and CUDA	6.7×10^{-8}
Polygon vertices through the solver	central FD	4.4×10^{-9} to 5.5×10^{-8}
Spline control points	central FD	1.8×10^{-8}
Modal $ S_{21} ^2$ objective, N ports	autograd, CPU and CUDA	1.3×10^{-15} to 5.4×10^{-15}
Far-field intensity objective	central FD	3.4×10^{-10}
Near-zone Poynting objective	central FD	2.7×10^{-9}
Streamed versus resident, tensor media	resident sweep	1×10^{-10}
Streamed versus resident, magnetic walls	resident sweep	3×10^{-7}

6.2 Gradient checks

We compare every differentiable path with full Torch automatic differentiation on a small grid and with central differences along a random direction, and we require the Taylor remainder of the directional derivative to fall quadratically. Table 2 lists the agreement, and the double-precision autograd comparisons differ at the round-off level. The material gradient of the 256^3 problem of Section 7 agrees with the complete-history reference to a relative L_2 error of 2.4×10^{-7} for the dielectric and 4.3×10^{-7} for the two-pole material, both in single precision. The transposed kernels therefore reproduce the derivative of the discrete forward model to round-off in double precision and to a few parts in 10^7 in single precision, in resident and streamed execution alike.

The discrete adjoint of a shape parameter converges to the physical shape derivative with the mesh. For a two-dimensional pentagon grating of permittivity 4 and period $0.6 \mu\text{m}$ at normal incidence and $1.55 \mu\text{m}$, the vertex gradient at mesh steps of 0.04, 0.02 and $0.01 \mu\text{m}$ differs from a central-difference reference at $0.005 \mu\text{m}$ by 11.6%, 1.8% and 0.5% for a $0.08 \mu\text{m}$ smoothing width. For a $0.04 \mu\text{m}$ width the two coarser steps give 5.4% and 1.0%. Halving the reference step moves the reference by less than 0.07%. On this grating a mesh step of $0.02 \mu\text{m}$ is therefore needed for the vertex gradient to approach the physical shape derivative within 2%.

6.3 Comparison with Meep, FDTD and a rigorous coupled-wave solver

To compare the package with independent implementations, we ran Meep 1.34 [7] on twelve MPI ranks of an i7-12700 and FDTD 0.6.2 [23] on the RTX 3060 of the same workstation, each from its own driver. The fixtures are the slab above, a Mie sphere at a $0.05 \mu\text{m}$ mesh, and the vacuum and sphere scenes of Table 5 at 64^3 and 96^3 cells. The absorbing layers share only their thickness, since FDTD grades its convolutional profile differently and Meep uses a stretched-coordinate layer. Only TorchFDTD closes a total-field/scattered-field box around the sphere, so the other two subtract an empty run from total-field planes. Meep runs in double precision and the GPU solvers in single precision. Between any two solvers the slab transmission differs by at most 1.6×10^{-5} and the sphere cross-sections by at most 0.7% (Table 3). The three codes agree more closely with each other than with the analytic slab response, so the remaining slab error belongs to the shared Yee discretization rather than to any one implementation.

On that device the fused engine steps 4.4 to 5.7 times faster than FDTD and completes a solve 6.4 to 7.3 times faster, and against the twelve Meep ranks, timed with other processes

Table 3: TorchFDTD, FDTD 0.6.2 and Meep 1.34 on one workstation with an i7-12700, on which Meep ran with twelve MPI ranks, and an RTX 3060 for the two GPU solvers. The slab columns are the maximum absolute errors of the transmission and reflection spectra against Eq. (5) over 31 wavelengths. The sphere column is the maximum relative cross-section error of a sphere of index 2.0 and radius $0.6 \mu\text{m}$ against the Mie series over nine wavelengths at a $0.05 \mu\text{m}$ mesh, with the source and monitor method of each solver. The last row is the largest difference between any two solvers. Meep propagates double-precision fields, the other two single precision.

Solver	Sphere source and monitors	Slab $ \Delta T _{\text{max}}$	Slab $ \Delta R _{\text{max}}$	Sphere error (%)
TorchFDTD	closed TFSF box, scattered-field planes	1.53×10^{-3}	1.54×10^{-3}	1.35
FDTD	one-way plane source, empty run subtracted	1.54×10^{-3}	1.53×10^{-3}	1.15
Meep	current sheet, empty run subtracted	1.55×10^{-3}	1.53×10^{-3}	1.15
Largest pairwise difference		1.60×10^{-5}	7.11×10^{-6}	0.68

Table 4: Forward throughput and time to gradient on the workstation of Table 3. The forward rows are the sphere scene at 800 steps as medians of three warmed solves, where the full solve includes setup and the final field transfer and the stepping column is the time-stepping loop alone. The adjoint rows are the permittivity gradient of a periodic dielectric slab at 64^3 cells and 128 steps, and the last column is the relative L_2 difference of each gradient from the TorchFDTD gradient. Meep ran in double precision on twelve MPI ranks with other processes present on the host, and its adjoint is excluded because it is a frequency-domain method. The GPU solvers ran in single precision.

Grid	Solver	Full solve (s)	Stepping (s)	Cell-steps/s stepping	Peak device (MiB)
64^3	TorchFDTD	0.114	0.094	2.22×10^9	18
64^3	FDTD	0.780	0.418	5.02×10^8	89
64^3	Meep	4.373	4.298	4.89×10^7	CPU
96^3	TorchFDTD	0.310	0.270	2.63×10^9	57
96^3	FDTD	1.998	1.507	4.70×10^8	288
96^3	Meep	13.602	13.356	5.30×10^7	CPU

Adjoint method	Time to gradient (s)	Peak device (MiB)	Gradient relative L_2 difference
TorchFDTD, checkpointed, 2 checkpoints	0.102	40	reference
FDTD, checkpointed, 2 checkpoints	5.268	303	7.1×10^{-8}
FDTD, reversible	0.209	111	1.0×10^{-7}

on the host, the full solve is 38 to 44 times faster (Table 4). For a periodic dielectric slab at 64^3 cells and 128 steps the checkpointed sweep returns the permittivity gradient 52 times faster than the FDTD checkpointed method with the same two checkpoints and 2.0 times faster than its reversible method, and the gradients agree to a relative L_2 difference of 1.1×10^{-7} . The Meep adjoint, a frequency-domain method, was not part of this comparison. Against FDTD the stepping ratio is smaller than the full-solve ratio, so part of the gain comes from setup and transfer rather than from the update kernels alone.

We also cross-checked the density path against TORCWA [21] on a device from our related work, the single-layer silicon nitride color router of Park et al. [44], a $2 \times 2 \mu\text{m}$ periodic cell observed at normal incidence between 420 and 670 nm. On a 15.6 nm mesh the fraction of incident power reaching each of its four detector wells differs from the TORCWA reference by at most 3.0 percentage points, with a mean difference of 1.2 points. At a continuous density

between the two materials, the adjoint derivative of a band-averaged routing objective agrees with central differences along two random directions to 1.3% and 0.07%.

6.4 Lateral tiling and exterior propagation

Because independent lateral tiles change the boundary-value problem, we measure their error against a full-domain reference. The tiling test uses a $140 \times 140 \times 60$ grid with 2059 time steps, split into four tiles with $3 \mu\text{m}$ cores (Fig. 4a,b). With the excitation sheet extended through the absorbers and hard core assembly, increasing the overlap from 0.25 to $2 \mu\text{m}$ reduces the six-component near-field relative L_2 error from 12.0% to 5.68%, while the total number of simulated cells grows to 2.9 times the full grid. In a separate gradient check at $1.5 \mu\text{m}$ overlap the focal objective differs by 0.138% from the full-domain value, whereas the design-region gradient differs by 27.2%. Consequently, agreement of the tiled derivative with a finite difference of its own forward model does not establish the accuracy of the full-device derivative, and we offer the tiled path for forward screening rather than final gradients.

Exterior propagation has a different error mechanism. For a fixed silicon-pillar lens the full observation-domain calculation uses $180 \times 180 \times 202$ cells, while FDTD to the output plane uses $180 \times 180 \times 62$ cells followed by Eq. (4). The relative intensity error over the axial section is 1.78% (Fig. 4c). Both routes locate the sampled maximum at $2.775 \mu\text{m}$ above the plane with transverse full widths at half maximum of 1.336 and $1.326 \mu\text{m}$. The long FDTD domain takes 60.5 s, the short domain 4.2 s and the FFT section 0.035 s on a shared RTX 3060, with 2412 and 2167 steps in the two FDTD runs (Fig. 4d). The two tests separate the coupling error of independent tiles from the sampling error of replacing a homogeneous exterior by propagation.

7 Performance

Unless stated otherwise, the timings below were taken on one NVIDIA RTX 5880 Ada with 48 GiB (51.5 GB) of device memory and 137 GB of host memory in single precision. Each value is the median of three warmed repetitions that include construction, graph capture, stepping and final-field transfer and exclude cold compilation. The cross-solver comparison of Section 6.3 and the propagated optical objective of Section 7.3 use the RTX 3060 workstation.

7.1 Forward throughput and ensembles

To isolate the effect of kernel fusion, we compare the fused engine with the PyTorch FDTD package it builds on, executed through one CUDA graph adapter so that both run without Python in the step loop [26]. Over eight fixtures at 64^3 and 96^3 cells and 800 steps the fused path completes a full solve 15.8 to 17.1 times faster at 64^3 and 9.6 to 12.1 times faster at 96^3 , with point-trace differences below 0.04% that arise from the different absorber stencils (Table 5). Sixteen-case parameter sweeps run 31 to 44 times faster than the sequential upstream package at 32^3 and 15 to 16 times faster at 64^3 when the cases share one tensor batch. Four sphere cases at 64^3 and 800 steps take 1.18 s with one CUDA worker against 47.0 s with one NumPy worker (Fig. 5). The tensor batch itself is 2.4 times faster than sequential native execution for sixteen 32^3 cases and slower than sequential execution for sixteen 64^3 cases, so cohorts of four are selected at that size (Table 6).

7.2 Resident and host-streamed adjoints

The memory–time trade-off of host streaming is isolated on matched 256^3 grids with 128 steps and two checkpoints (Fig. 6). Timings run from model construction to the transfer of material

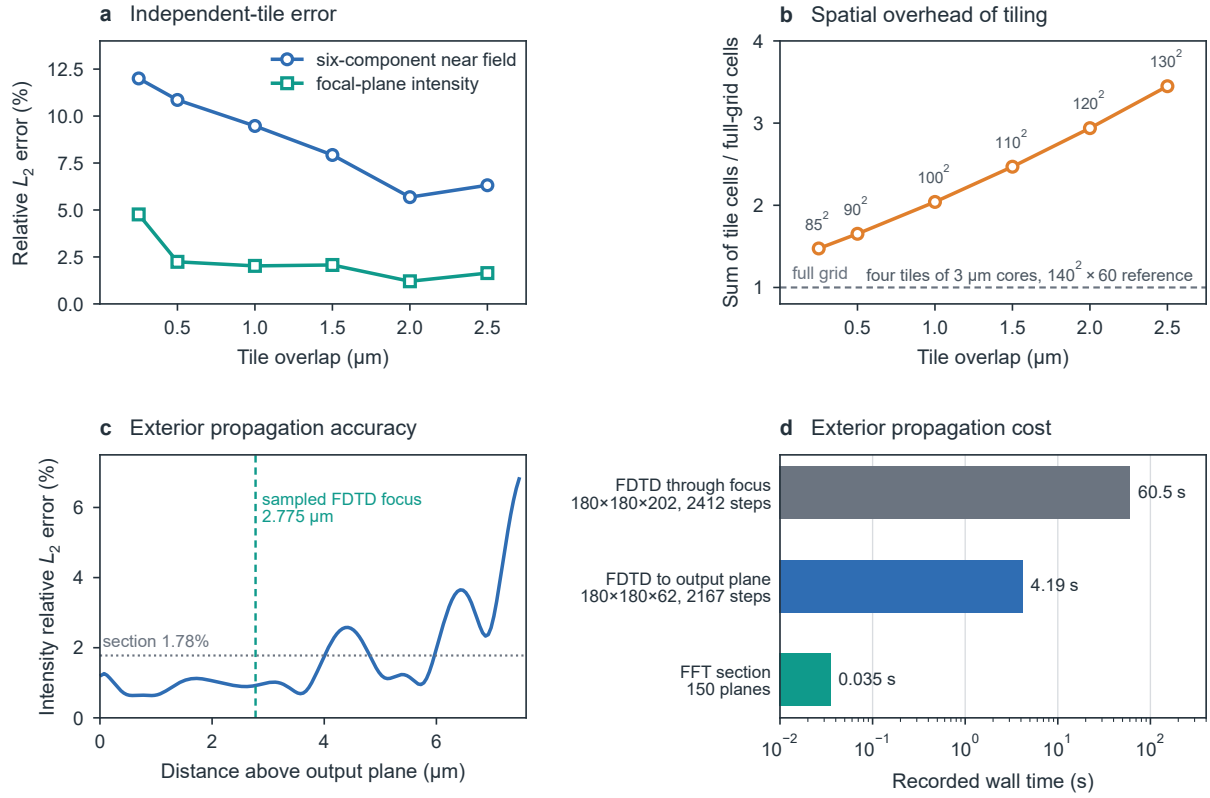


Figure 4: Accuracy and computational extent of the two spatial reductions on an RTX 3060. (a) Full-domain reference errors for independent overlapping tiles with hard core assembly. (b) The summed cell count of all tiles relative to the full grid, including repeated overlap regions. (c) Relative intensity error at each axial position for angular-spectrum propagation compared with FDTD through the observation region. The dashed line marks the sampled FDTD focus. (d) The recorded FDTD and FFT wall times on a logarithmic axis. Panels (a,b) and (c,d) use different lens fixtures.

Table 5: Eight forward fixtures on RTX 5880. The external baseline is flaport/fdtd 0.2.2 with an added CUDA Graph adapter. Values are medians of three warmed full solves. The last column is point-trace relative L2 difference in percent, not analytic error. The PML interface stencils differ and final fields are not identical.

Fixture, grid	Upstream graph (ms)	Native fused (ms)	Speedup	Trace L2 (%)
Vacuum, 64 ³	649.23	38.06	17.06×	0.0142
Sphere, 64 ³	629.07	39.72	15.84×	0.0233
Slab, 64 ³	636.26	37.23	17.09×	0.0197
Waveguide, 64 ³	632.59	39.90	15.85×	0.0372
Vacuum, 96 ³	954.98	78.77	12.12×	0.0120
Sphere, 96 ³	848.58	88.05	9.64×	0.0208
Slab, 96 ³	836.66	80.37	10.41×	0.0174
Waveguide, 96 ³	848.07	80.00	10.60×	0.0328

Table 6: Native sphere ensembles on RTX 5880 with a real CUDA batch axis. Full wall times include setup, transfers and scalar objective evaluation. Final E/H arrays and point traces match independent native runs bitwise. Values below unity in the last column are slowdowns.

Grid, cases	Sequential (ms)	Tensor cohort (ms)	Cases/s	Speedup
32^3 , 1	23.22	20.38	49.06	1.14×
32^3 , 2	46.00	31.39	63.71	1.47×
32^3 , 4	106.81	63.67	62.83	1.68×
32^3 , 8	181.89	88.24	90.66	2.06×
32^3 , 16	341.29	142.46	112.31	2.40×
64^3 , 1	39.08	40.49	24.70	0.97×
64^3 , 2	86.40	78.29	25.55	1.10×
64^3 , 4	167.68	153.87	26.00	1.09×
64^3 , 8	307.35	357.10	22.40	0.86×
64^3 , 16	669.53	852.94	18.76	0.78×

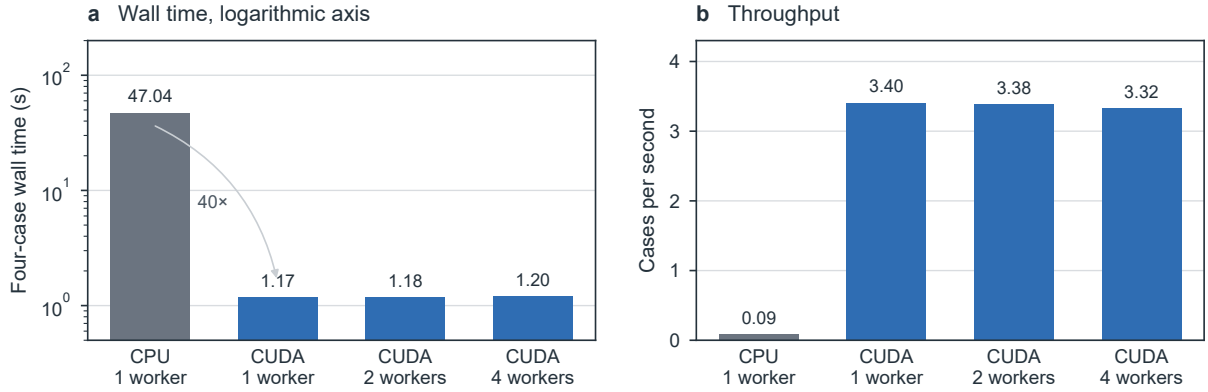


Figure 5: Wall time on a logarithmic axis (a) and throughput (b) of four-structure ensembles. Bars are medians of three measured batches after warm-up.

cotangents to the host. Specifically, the dielectric case takes 2.28 s resident on the GPU and 9.43 s with host-bank streaming, and the two-pole case takes 5.00 s and 23.90 s, while the corresponding device peaks decrease from 2.02 to 0.90 GB and from 5.51 to 2.41 GB. Streaming therefore lowers the device allocation by about 56% at a factor of 4.1 to 4.8 in elapsed time for these policies, which places it as the mode for problems whose resident allocation would not fit rather than as an acceleration of a resident adjoint. For context, this Torch model on twelve CPU threads takes 407.7 s and 742.0 s, so the resident GPU is 179 and 149 times faster on these fixtures and host streaming retains factors of 43 and 31.

7.3 A propagated optical objective on a workstation

To exercise host streaming on a spatially extended optical objective, we evaluate a fixed array of 1936 dielectric pillars over a $44 \times 44 \mu\text{m}$ footprint. The pillars have relative permittivity 4, height $0.6 \mu\text{m}$, period $1 \mu\text{m}$ and radii between 0.15 and $0.42 \mu\text{m}$, and a $0.05 \mu\text{m}$ mesh gives $920 \times 920 \times 64$ cells. A plane-wave pulse is propagated for 1836 steps, or 175 fs, by which time the output-plane electric energy has decayed to 5.91×10^{-4} of its recorded peak. The objective sums the on-axis intensity at 1.45 , 1.55 and $1.65 \mu\text{m}$ evaluated $150 \mu\text{m}$ beyond the output plane with Eq. (4). The plane spectra are normalized by the sampled time interval, spatially

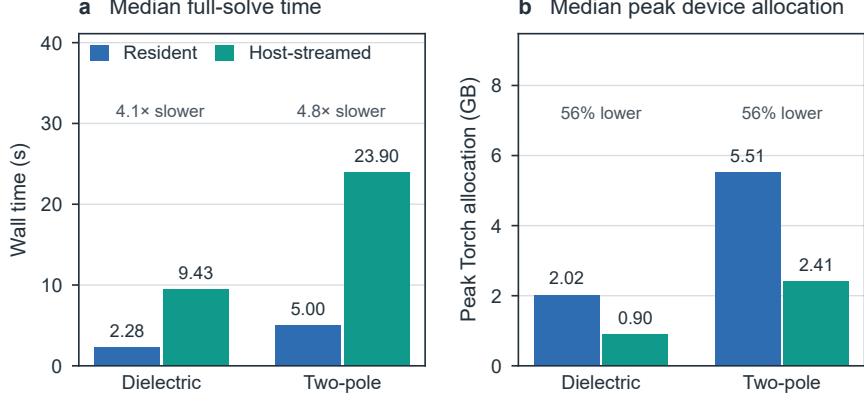


Figure 6: Memory–time trade-off between GPU-resident and host-streamed adjoints on the RTX 5880. Matched 256^3 grids use 128 steps and two checkpoints. (a) Median full-solve time. (b) Median peak Torch device allocation, which excludes the CUDA context and non-Torch allocations. Both quantities use three interleaved warmed repetitions. GB denotes 10^9 bytes.

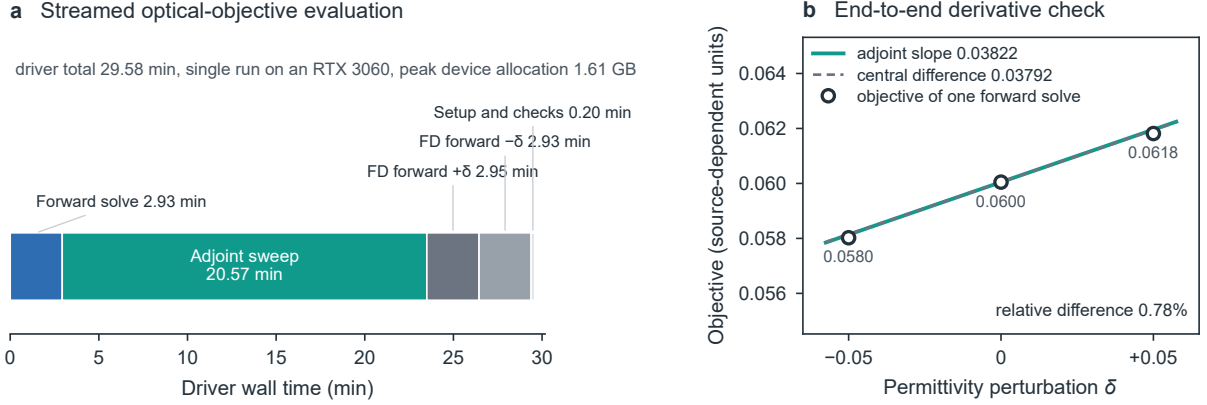


Figure 7: Streamed evaluation of the fixed $44 \mu\text{m}$ pillar-array objective on an RTX 3060. (a) Timeline of the recorded driver: the forward pass, the adjoint sweep, the two central-difference (FD) forward solves and the remaining setup and checks. (b) The objective at the unperturbed and the two perturbed permittivities of the central pillar region. The adjoint directional derivative and the central-difference secant are drawn as slopes through the unperturbed value. Both use the same sampled output plane and propagation model.

downsampled by eight to 110×110 points and propagated with a padding factor of three, and the same sampling is used for the adjoint and the finite differences.

On the RTX 3060, host-bank streaming with slab width 64, temporal depth 32, six global checkpoints and one local checkpoint completes the forward and reverse passes in 176.0 s and 1234.0 s (Fig. 7a). The peak device allocation during the reverse pass is 1.61 GB and the sampled process resident memory is 22.19 GB. Two further forward solves evaluate a central difference with a permittivity perturbation of 0.05 in the central pillar region. The adjoint and central-difference directional derivatives are 0.0382185 and 0.0379211, a relative difference of 0.78% (Fig. 7b). The complete driver, including those checks, takes 29.58 minutes in a single run. The example connects propagation through the structured region, a remote optical objective and a dense material derivative without meshing the intervening exterior. Its 1.30 GB electric and magnetic state fits the device, so the example measures the streamed path at a reduced device footprint rather than beyond the device capacity.

8 Conclusion

We have shown that the discrete adjoint of fused CUDA time stepping can be exposed to PyTorch and executed with its field state in host memory, so that a single workstation GPU returns material and geometry derivatives of full-wave photonic objectives. The transposed kernels of TorchFDTD give the gradient of differentiable Torch objectives built from the supported observations with respect to permittivity fields, densities, analytic solids, outlines and source waveforms, and its causal slab streaming bounds the device allocation by one slab and its halos while leaving the discrete problem unchanged. The package agrees with analytic solutions, Meep, FDTDx and a rigorous coupled-wave solver, and its adjoint matches automatic differentiation and finite differences. On the tested 64^3 and 96^3 scenes its full forward solves are 9.6 to 17.1 times faster than those of the package it extends on an RTX 5880 and 6.4 to 7.3 times faster than those of FDTDx on an RTX 3060. The software is available under the MIT license [45].

Most remaining questions are architectural. First, host streaming keeps dense material arrays, state banks and checkpoints in host memory, so sustained optimization of a device whose field state exceeds the GPU will be governed by host capacity and device–host bandwidth, and the multi-GPU execution of one domain is the next step. Second, the tiled forward path invites a coupled formulation that recovers the lateral coupling that independent tiles discard. On the numerical side, dispersive media that cross an absorbing face and conformal treatment of curved metallic surfaces remain open. Within these bounds, the time-domain adjoint of a photonic device with tens of millions of cells can be evaluated on the hardware that a design group owns.

Declaration of competing interest

The author declares no competing financial interests or personal relationships that could have influenced the work reported in this paper.

Declaration of generative AI and AI-assisted technologies

Software development. During the development of TorchFDTD the author used generative AI tools (OpenAI GPT-6 Astra and Anthropic Claude Fable 5.1) for assistance with implementation, test generation and benchmark scripting. The resulting code was checked by the automated test suite and by the numerical validation reported in this work.

Manuscript preparation. During the preparation of this manuscript the author used Anthropic Claude (Fable 5.1 and Opus 5.5) to improve the language, style and readability of the text. After using these tools the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Data availability

The source code, the test suite and the complete set of validation records are available in the public repository [45]. The ancillary files of this preprint contain the records behind every figure and generated table, the two scripts that regenerate those figures and tables from the records, and their SHA-256 provenance manifests. Table 2 and the remaining in-text values are taken from records in the repository. The drivers of the color-router cross-check are in the ancillary files, and the device model they require is in the repository of Ref. [44].

References

- [1] Kane S. Yee. Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media. *IEEE Transactions on Antennas and Propagation*, 14(3): 302–307, 1966. doi: 10.1109/TAP.1966.1138693.
- [2] Allen Taflov and Susan C. Hagness. *Computational Electrodynamics: The Finite-Difference Time-Domain Method*. Artech House, Norwood, MA, 3 edition, 2005.
- [3] Jean-Pierre Bérenger. A perfectly matched layer for the absorption of electromagnetic waves. *Journal of Computational Physics*, 114(2):185–200, 1994. doi: 10.1006/jcph.1994.1159.
- [4] J. Alan Roden and Stephen D. Gedney. Convolution PML (CPML): An efficient FDTD implementation of the CFS-PML for arbitrary media. *Microwave and Optical Technology Letters*, 27(5):334–339, 2000.
- [5] Michal Okoniewski, Michal Mrozowski, and Maria A. Stuchly. Simple treatment of multi-term dispersion in FDTD. *IEEE Microwave and Guided Wave Letters*, 7(5):121–123, 1997. doi: 10.1109/75.569723.
- [6] Ardavan Farjadpour, David Roundy, Alejandro Rodriguez, Mihai Ibanescu, Peter Bermel, John D. Joannopoulos, Steven G. Johnson, and Geoffrey W. Burr. Improving accuracy by subpixel smoothing in the finite-difference time domain. *Optics Letters*, 31(20):2972–2974, 2006. doi: 10.1364/OL.31.002972.
- [7] Ardavan F. Oskooi, David Roundy, Mihai Ibanescu, Peter Bermel, John D. Joannopoulos, and Steven G. Johnson. Meep: A flexible free-software package for electromagnetic simulations by the FDTD method. *Computer Physics Communications*, 181(3):687–702, 2010. doi: 10.1016/j.cpc.2009.11.008.
- [8] Ansys. Ansys Lumerical FDTD. Software, 2025. URL <https://www.ansys.com/products/optics/fdtd>. Accessed 21 September 2026.
- [9] Flexcompute. Tidy3D: hardware-accelerated electromagnetic solver. Software, 2025. URL <https://www.flexcompute.com/tidy3d/>. Accessed 21 September 2026.
- [10] Natalia K. Georgieva, Snezana Glavic, Mohamed H. Bakr, and John W. Bandler. Feasible adjoint sensitivity technique for EM design optimization. *IEEE Transactions on Microwave Theory and Techniques*, 50(12):2751–2758, 2002. doi: 10.1109/TMTT.2002.805131.
- [11] Jakob S. Jensen and Ole Sigmund. Topology optimization for nano-photonics. *Laser & Photonics Reviews*, 5(2):308–321, 2011. doi: 10.1002/lpor.201000014.
- [12] Yuriy Elesin, Boyan S. Lazarov, Jakob S. Jensen, and Ole Sigmund. Design of robust and efficient photonic switches using topology optimization. *Photonics and Nanostructures - Fundamentals and Applications*, 10(1):153–165, 2012. doi: 10.1016/j.photonics.2011.10.003.
- [13] Alexander Y. Piggott, Jesse Lu, Konstantinos G. Lagoudakis, Jan Petykiewicz, Thomas M. Babinec, and Jelena Vučković. Inverse design and demonstration of a compact and broadband on-chip wavelength demultiplexer. *Nature Photonics*, 9:374–377, 2015. doi: 10.1038/nphoton.2015.69.
- [14] Sean Molesky, Zin Lin, Alexander Y. Piggott, Weiliang Jin, Jelena Vučković, and Alejandro W. Rodriguez. Inverse design in nanophotonics. *Nature Photonics*, 12:659–670, 2018. doi: 10.1038/s41566-018-0246-9.
- [15] Tyler W. Hughes, Momchil Minkov, Ian A. D. Williamson, and Shanhui Fan. Adjoint method and inverse design for nonlinear nanophotonic devices. *ACS Photonics*, 5(12):4781–4787, 2018. doi: 10.1021/acsp Photonics.8b01522.
- [16] Rasmus E. Christiansen and Ole Sigmund. Inverse design in photonics by topology optimization: tutorial. *Journal of the Optical Society of America B*, 38(2):496–509, 2021. doi: 10.1364/JOSAB.406048.

- [17] Logan Su, Dries Vercruysse, Jinjie Skarda, Neil V. Sapro, Jan A. Petykiewicz, and Jelena Vučković. Nanophotonic inverse design with SPINS: Software architecture and practical considerations. *Applied Physics Reviews*, 7(1):011407, 2020. doi: 10.1063/1.5131263.
- [18] Momchil Minkov, Ian A. D. Williamson, Lucio C. Andreani, Dario Gerace, Beicheng Lu, and Shanhui Fan. Inverse design of photonic crystals through automatic differentiation. *ACS Photonics*, 7(7):1729–1741, 2020. doi: 10.1021/acsp Photonics.0c00327.
- [19] Alec M. Hammond, Ardavan Oskooi, Mo Chen, Zin Lin, Steven G. Johnson, and Stephen E. Ralph. High-performance hybrid time/frequency-domain topology optimization for large-scale photonics inverse design. *Optics Express*, 30(3):4467–4491, 2022. doi: 10.1364/OE.442074.
- [20] Tyler W. Hughes, Ian A. D. Williamson, Momchil Minkov, and Shanhui Fan. Forward-mode differentiation of Maxwell’s equations. *ACS Photonics*, 6(11):3010–3016, 2019. doi: 10.1021/acsp Photonics.9b01238.
- [21] Changhyun Kim and Byoungcho Lee. TORCWA: GPU-accelerated Fourier modal method and gradient-based optimization for metasurface design. *Computer Physics Communications*, 282:108552, 2023. doi: 10.1016/j.cpc.2022.108552.
- [22] Jesse Lu. fdtdz: A fast FDTD solver for photonics on GPUs. Software, 2023. URL <https://github.com/spinsphotonics/fdtdz>. Accessed 21 September 2026.
- [23] Yannik Mahlau, Frederik Schubert, Konrad Bethmann, Reinhard Caspary, Antonio Calà Lesina, Marco Munderloh, Jörn Ostermann, and Bodo Rosenhahn. A flexible framework for large-scale FDTD simulations: open-source inverse design for 3D nanostructures. arXiv:2412.12360, 2024.
- [24] Andreas Griewank and Andrea Walther. Algorithm 799: revolve: an implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software*, 26(1):19–45, 2000. doi: 10.1145/347837.347846.
- [25] Yannik Mahlau, Lukas Berg, and Bodo Rosenhahn. Numerical field optimization for enhanced efficiency in time-reversible gradient computation of open-source GPU-accelerated FDTD simulations. arXiv:2603.24027, 2026.
- [26] Floris Laporte et al. fdtd: A three-dimensional electromagnetic FDTD simulator in Python, version 0.2.2. MIT-licensed software, 2023. URL <https://github.com/flaport/fdtd>. Accessed 19 September 2026.
- [27] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.
- [28] Ardavan F. Oskooi, Chris Kottke, and Steven G. Johnson. Accurate finite-difference time-domain simulation of anisotropic media by subpixel smoothing. *Optics Letters*, 34(18):2778–2780, 2009. doi: 10.1364/OL.34.002778.
- [29] Eliane Bécache, Sandrine Fauqueux, and Patrick Joly. Stability of perfectly matched layers, group velocities and anisotropic waves. *Journal of Computational Physics*, 188(2):399–433, 2003. doi: 10.1016/S0021-9991(03)00184-0.
- [30] Ardavan Oskooi and Steven G. Johnson. Distinguishing correct from incorrect PML proposals and a corrected unsplit PML for anisotropic, dispersive media. *Journal of Computational Physics*, 230(7):2369–2377, 2011. doi: 10.1016/j.jcp.2011.01.006.
- [31] Lucas Heitzmann Gabrielli. gdstk: GDSII and OASIS library for Python and C++. Software, 2024. URL <https://github.com/heitzmann/gdstk>. Accessed 21 September 2026.

- [32] John B. Schneider. *Understanding the Finite-Difference Time-Domain Method*. School of Electrical Engineering and Computer Science, Washington State University, 2010. <https://www.eecs.wsu.edu/~schneidj/ufdtd/>.
- [33] Zhaoming Zhu and Thomas G. Brown. Full-vectorial finite-difference analysis of microstructured optical fibers. *Optics Express*, 10(17):853–864, 2002. doi: 10.1364/OE.10.000853.
- [34] Raymond J. Luebbers, Karl S. Kunz, Michael Schneider, and Forrest Hunsberger. A finite-difference time-domain near zone to far zone transformation. *IEEE Transactions on Antennas and Propagation*, 39(4):429–433, 1991. doi: 10.1109/8.81453.
- [35] Boyan S. Lazarov and Ole Sigmund. Filters in topology optimization based on Helmholtz-type differential equations. *International Journal for Numerical Methods in Engineering*, 86(6):765–781, 2011. doi: 10.1002/nme.3072.
- [36] Fengwen Wang, Boyan S. Lazarov, and Ole Sigmund. On projection methods, convergence and robust formulations in topology optimization. *Structural and Multidisciplinary Optimization*, 43(6):767–784, 2011. doi: 10.1007/s00158-010-0602-y.
- [37] Ryosuke Okuta, Yuya Unno, Daisuke Nishino, Shohei Hido, and Crissman Loomis. CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In *Proceedings of the Workshop on Machine Learning Systems (LearningSys) at the 31st Conference on Neural Information Processing Systems*, 2017.
- [38] Rainer Storn and Kenneth Price. Differential evolution: A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997. doi: 10.1023/A:1008202821328.
- [39] Raphaël Pestourie, Carlos Pérez-Arancibia, Zin Lin, Wonseok Shin, Federico Capasso, and Steven G. Johnson. Inverse design of large-area metasurfaces. *Optics Express*, 26(26):33732–33747, 2018. doi: 10.1364/OE.26.033732.
- [40] Zin Lin and Steven G. Johnson. Overlapping domains for topology optimization of large-area metasurfaces. *Optics Express*, 27(22):32445–32453, 2019. doi: 10.1364/OE.27.032445.
- [41] Joseph W. Goodman. *Introduction to Fourier Optics*. Roberts and Company Publishers, 3 edition, 2005.
- [42] Kyoji Matsushima and Tomoyoshi Shimobaba. Band-limited angular spectrum method for numerical simulation of free-space propagation in far and near fields. *Optics Express*, 17(22):19662–19673, 2009. doi: 10.1364/OE.17.019662.
- [43] Craig F. Bohren and Donald R. Huffman. *Absorption and Scattering of Light by Small Particles*. Wiley, New York, 1983.
- [44] Hyoseok Park, Sehyeon Park, Myungjae Lee, and Yeonsang Park. Imaging-system-aware color routers optimized for imaging information. arXiv:2608.13019, 2026.
- [45] Hyoseok Park. TorchFDTD source code and validation records. GitHub repository, MIT licence, 2026. URL <https://github.com/hyoseokp/TorchFDTD>. Accessed 21 September 2026.